CubeMX Setup: IMU Test Project

STM32F407VET6 on the Hiwonder ROS Robot Controller V1.2 · project **ImuBoardTest** · MTR 520 2DOF Drone Rig

> **Goal:** read the on-board IMU (QMI8658 at I2C address 0x6A) and print its values to a laptop. Needs only the clock, I2C2, USART3 and the LED. The encoder timers (TIM5, TIM2) are included because they come next; they stay idle until code starts them.

1 Create the project

File → New Project → in Commercial Part Number type **STM32F407VET6** → select it → **Start Project**.

Moving a pin: type the pin name in the search box under the chip picture, left-click the highlighted pin and choose the function (e.g. TIM2_CH1). CubeMX moves that function off its old pin.

2 System Core

Item	Setting
RCC	High Speed Clock (HSE) = **Crystal/Ceramic Resonator** (the board has a 16 MHz crystal)
SYS	Debug = **Serial Wire** (keeps PA13/PA14 for SWD and frees PA15, PB3, PB4)

3 Clock Configuration tab

Box	Value
Input frequency (HSE)	**16** MHz
PLL Source Mux	**HSE**
System Clock Mux	**PLLCLK**
HCLK	type **168** and press Enter; accept the automatic solver
Check	APB1 timer clocks = **84 MHz**, APB2 timer clocks = **168 MHz**

4 Connectivity (set I2C2 first, then USART3)

Periph.	Mode	Pins	Parameter Settings
I2C2	I2C	**PB10** SCL, **PB11** SDA	I2C Speed Mode = **Fast Mode** (400 kHz)
USART3	Asynchronous	⚠ move to **PD8** TX, **PD9** RX	Baud **115200**, 8 bits, no parity, 1 stop. Default pins PB10/PB11 would clash with I2C2

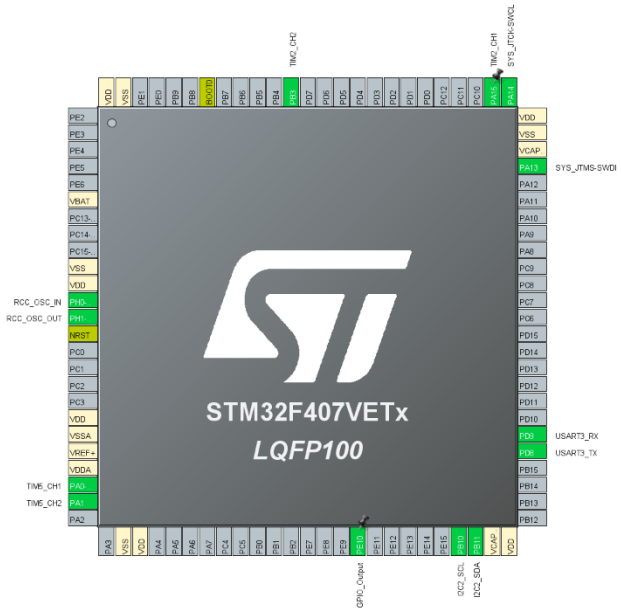
5 Timers (encoders, for the next test)

Timer	Combined Channels	Pins	Parameter Settings
TIM5	Encoder Mode	**PA0**, **PA1**	Counter Period **4294967295**, Encoder Mode **TI1 and TI2**
TIM2	Encoder Mode	⚠ move to **PA15**, **PB3**	Same as TIM5. Default PA0 would clash with TIM5

6 GPIO

Pin	Function	Setting (System Core → GPIO)
PE10	GPIO_Output	Output level **High** (LED off at start; the LED lights when the pin is LOW). Optional user label **LED**

7 Check the pinout



All pins must be **green**, with no orange or red warnings on the left panel.

Pin	Function
PH0, PH1	RCC_OSC_IN / OUT
PA13, PA14	SWDIO / SWCLK
PA0, PA1	TIM5_CH1 / CH2
PA15, PB3	TIM2_CH1 / CH2
PB10, PB11	I2C2_SCL / SDA
PD8, PD9	USART3_TX / RX
PE10	GPIO_Output (LED)

8 Generate the code

Project Manager tab: Project Name **ImuBoardTest**, choose a folder, Toolchain / IDE = **STM32CubeIDE**. Recommended: **Code Generator → tick "Generate peripheral initialization as a pair of '.c/.h' files per peripheral"**. This moves the setup code out of main.c into i2c.c, usart.c, tim.c and gpio.c.
Click **GENERATE CODE → Open Project**.

9 Add the project code in STM32CubeIDE

File	Folder	What it does
imu.h / imu.c	Core/Inc, Core/Src	IMU driver: bus scan, chip detection (QMI8658 or MPU6050), reading
app.h / app.c	Core/Inc, Core/Src	Application: App_Init() and App_Loop()
retarget.c	Core/Src	Sends printf() to USART3 (the UART2 USB-C port)

In **Core/Src/main.c** add only three lines, each inside its USER CODE section:

main.c section	Line to add
USER CODE BEGIN Includes	**#include "app.h"**
USER CODE BEGIN 2	**App_Init();**
USER CODE BEGIN 3 (inside while (1))	**App_Loop();**

Then: Project → Properties → C/C++ Build → Settings → MCU Settings → tick **Use float with printf from newlib-nano** → Apply. Build with the **hammer** icon (0 errors).

> **Keep code inside USER CODE sections** (or in your own files). When the .ioc is changed and code is regenerated, CubeMX keeps those sections and overwrites everything else. Loading and running: see **How_to_Load_and_Run.pdf**.